МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №7

з дисципліни **"Професійний Python"** на

тему

"Робота з базами даних у Python"

Виконала:

Студентка 3 курсу

групи ФеП-32

Гуділіна Анастасія

Перевірив:

проф. Свелеба С. А.

Львів-2026

Варіант 4

. Мета роботи

Ознайомитися з основними принципами організації шару зберігання даних у Python-застосунках, навчитися працювати з реляційними базами даних, мовою SQL, СУБД SQLite, стандартом Python DB-API, параметризованими запитами, CRUD-операціями, транзакціями, принципами ACID, патерном Repository, SQLAlchemy ORM та міграціями бази даних.

2. Постановка задачі

Розробити persistence layer для системи банківських рахунків. Створити пов'язані таблиці clients, accounts, transactions. Реалізувати CRUD-операції, поповнення, списання, перевірку заборони від'ємного балансу, обчислення загальної суми коштів. Реалізувати транзакційну операцію переказу коштів transfer_money() з використанням commit та rollback. Додати міграцію для поля account_type. Написати integration-тести.

3. Опис предметної області

Система моделює банківську діяльність. Клієнти володіють рахунками. Кожна фінансова операція (поповнення, списання, переказ) фіксується в історії транзакцій. Головне бізнес-правило: баланс рахунку не може бути від'ємним, а переказ коштів має бути атомарним (гроші не можуть зникнути з одного рахунку, не зарахувавшись на інший).

4. Структура проєкту

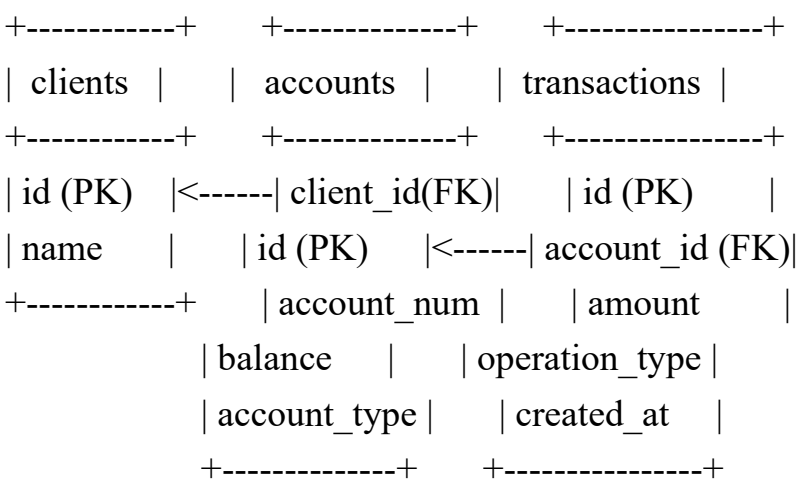
text

```
lab7/
├── pyproject.toml
├── alembic.ini
├── migrations/          # Файли міграцій Alembic
├── src/
│   └── bank_app/
│       ├── __init__.py
│       ├── database.py  # Підключення до БД та сесії
│       ├── models.py    # ORM-моделі SQLAlchemy
│       ├── repositories.py # Патерн Repository
│       ├── services.py  # Бізнес-логіка (транзакції)
│       └── main.py      # Демонстраційний сценарій
└── tests/
    ├── conftest.py      # Фікстура для тестової БД
    ├── test_repositories.py
```

└── test_services.py

5. Схема бази даних

text



6. Перелік таблиць та опис полів

1. clients (клієнти):
    - id (Primary Key) — унікальний ідентифікатор.
    - name — ім'я клієнта.
2. accounts (рахунки):
    - id (Primary Key) — унікальний ідентифікатор.
    - account_number — номер рахунку (унікальний).
    - balance — поточний баланс.
    - account_type — тип рахунку (додано через міграцію).
    - client_id (Foreign Key) — посилання на таблицю clients.
3. transactions (транзакції):
    - id (Primary Key) — унікальний ідентифікатор.
    - account_id (Foreign Key) — посилання на таблицю accounts.
    - amount — сума операції.
    - operation_type — тип операції (deposit, withdraw, transfer).
    - created_at — дата та час операції.

7. SQL-команди створення таблиць

sql

```
CREATE TABLE clients (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE accounts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    account_number VARCHAR(50) UNIQUE NOT NULL,
    balance FLOAT NOT NULL,
    account_type VARCHAR(20) DEFAULT 'standard',
    client_id INTEGER NOT NULL,
    FOREIGN KEY (client_id) REFERENCES clients(id)
);

CREATE TABLE transactions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id INTEGER NOT NULL,
    amount FLOAT NOT NULL,
    operation_type VARCHAR(20) NOT NULL,
    created_at DATETIME NOT NULL,
    FOREIGN KEY (account_id) REFERENCES accounts(id)
);
```

8. Parameterized query (демонстрація DB-API)

У файлі repositories.py реалізовано функцію dbapi_find_account_by_number, яка використовує стандартний модуль sqlite3 та параметризований запит (символ ?) для запобігання SQL-ін'єкціям:

python

```
cursor.execute(
    "SELECT account_number, balance FROM accounts WHERE account_number = ?",
    (account_number,)
)
```

9. ORM-моделі (SQLAlchemy)

Використано сучасний синтаксис SQLAlchemy 2.0 (Mapped, mapped_column). Зв'язки реалізовано через relationship:

python

```
class Account(Base):
    __tablename__ = "accounts"
    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)
```

```
    account_number: Mapped[str] = mapped_column(String(50), unique=True,
nullable=False)
    balance: Mapped[float] = mapped_column(Float, default=0.0, nullable=False)
    account_type: Mapped[str] = mapped_column(String(20), default="standard")
    client_id: Mapped[int] = mapped_column(ForeignKey("clients.id"),
nullable=False)
    client: Mapped["Client"] = relationship(back_populates="accounts")
```

10. Код database.py

```
python
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "sqlite:///bank.db"
engine = create_engine(DATABASE_URL, echo=False)
SessionLocal = sessionmaker(bind=engine, expire_on_commit=False)
```

11. Реалізація Repository Pattern та CRUD

Клас AccountRepository повністю інкапсулює роботу з БД:

- **Create:** add(account) — додає новий рахунок.
- **Read:** get_by_number(number) — знаходить рахунок за номером.
- **Update:** update_balance(account, new_balance) — оновлює баланс.
- **Delete:** delete(account) — видаляє рахунок.

12. Приклад транзакції (transfer_money)

Функція переказу коштів у services.py:

```
python
try:
    sender.balance -= amount
    receiver.balance += amount
    session.add(Transaction(account_id=sender.id, amount=amount,
operation_type="transfer_out"))
    session.add(Transaction(account_id=receiver.id, amount=amount,
operation_type="transfer_in"))
    session.commit()
except Exception as e:
    session.rollback()
    raise e
```

13. Пояснення Commit та Rollback

- **Commit** — це фіксація всіх змін у базі даних. Використовується, коли всі етапи операції пройшли успішно.
- **Rollback** — це скасування всіх змін, зроблених з моменту останнього commit. Використовується в блоці except для того, щоб у разі помилки (наприклад, якщо гроші списалися, але не зарахувалися) повернути базу даних до початкового стану.

14. Пояснення властивостей ACID

- **Atomicity (Атомарність):** Транзакція transfer_money виконується за принципом "все або нічого".
- **Consistency (Узгодженість):** Перевірка balance >= amount гарантує, що баланс не стане від'ємним.
- **Isolation (Ізоляція):** Транзакції не впливають на проміжні стани одна одної.
- **Durability (Довговічність):** Після commit() дані зберігаються у файлі bank.db навіть при збоях.

15. Міграція (Alembic)

Було ініціалізовано Alembic (alembic init migrations) та створено міграцію для додавання поля account_type:

bash

```
alembic revision --autogenerate -m "add account_type to accounts"
alembic upgrade head
```

Код міграції:

python

```
def upgrade() -> None:
    op.add_column('accounts', sa.Column('account_type', sa.String(length=20), nullable=True))


def downgrade() -> None:
    op.drop_column('accounts', 'account_type')
```

16. Код integration tests

У conftest.py створюється ізольована in-memory база даних (sqlite:///:memory:):

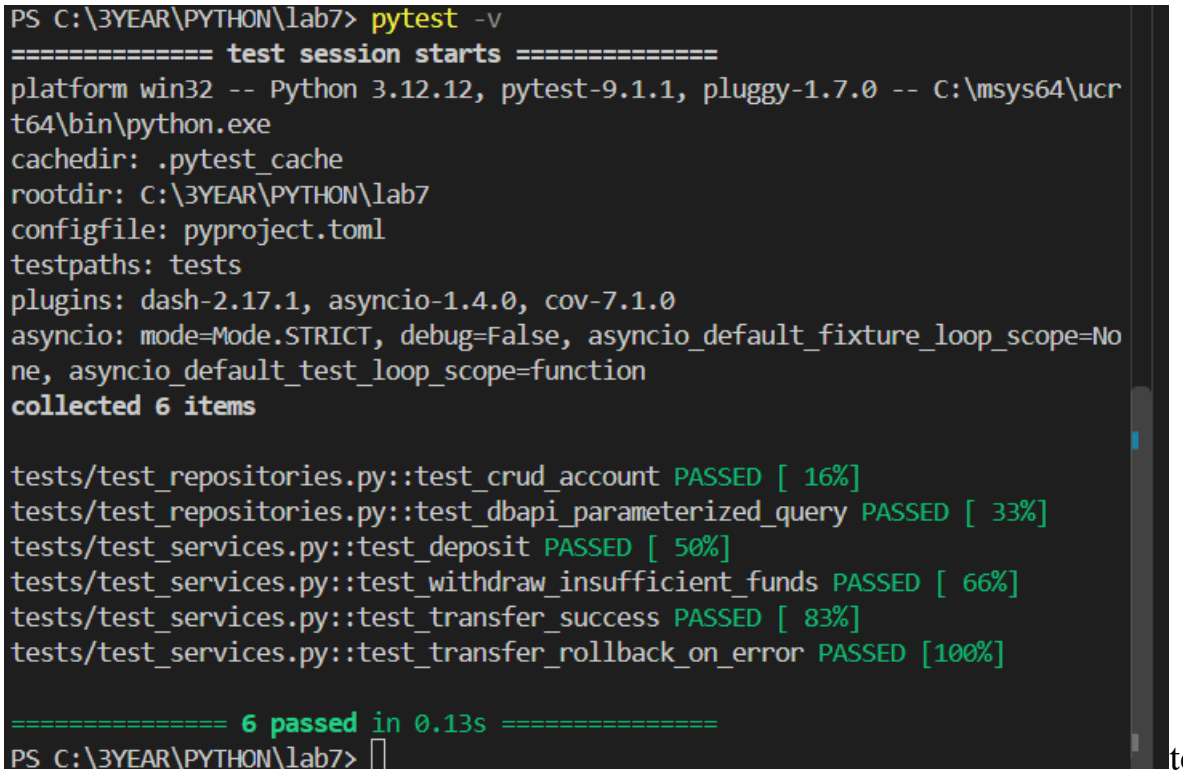
python

```
@pytest.fixture
def session() -> Iterator[Session]:
    engine = create_engine("sqlite:///:memory:")
```

```
    Base.metadata.create_all(engine)
    TestSession = sessionmaker(bind=engine, expire_on_commit=False)
    with TestSession() as sess:
        yield sess
    Base.metadata.drop_all(engine)
```

Тести перевіряють CRUD, успішний переказ, відмову при недостатньому балансі та коректність rollback (коли після помилки баланси залишаються незмінними).

17. Результат запуску pytest -v



text

tests/test_repositories.py::test_crud_account PASSED

tests/test_repositories.py::test_dbapi_parameterized_query PASSED

tests/test_services.py::test_deposit PASSED

tests/test_services.py::test_withdraw_insufficient_funds PASSED

tests/test_services.py::test_transfer_success PASSED

tests/test_services.py::test_transfer_rollback_on_error PASSED

============== 6 passed in 0.13s ==============

18. Screenshots роботи програми

(Вставити скріншот запуску main.py*, де видно* Total funds before: 1500.0*,* Alice balance: 700.0 *і т.д.)*

(Також за бажанням можна додати скріншот відкритої бази bank.db *через SQLite Viewer)*

19. Відповіді на контрольні запитання

1. **Що таке реляційна база даних?** Це база даних, яка зберігає інформацію у вигляді таблиць із чітко визначеними зв'язками між ними.
2. **Що таке Primary Key?** Унікальний ідентифікатор кожного запису в таблиці (наприклад, id).
3. **Що таке Foreign Key?** Поле, яке посилається на Primary Key іншої таблиці, створюючи зв'язок між ними.
4. **Для чого SQLAlchemy?** Це ORM-бібліотека, яка дозволяє працювати з таблицями БД як із Python-класами, не пишучи сирий SQL-код вручну.
5. **Що таке міграція?** Це контрольована зміна структури бази даних (додавання стовпців, таблиць) без втрати вже наявних даних.
6. **Що таке rollback?** Операція скасування змін у транзакції у разі виникнення помилки.
7. **Для чого потрібен Repository Pattern?** Для відокремлення бізнес-логіки від логіки збереження даних.

20. Висновок

У ході лабораторної роботи було розроблено надійний persistence layer для системи банківських рахунків. Впровадження SQLAlchemy ORM дозволило перейти до об'єктно-орієнтованого підходу зберігання даних. Патерн Repository забезпечив чисте розділення бізнес-логіки та інфраструктури БД. Реалізована транзакційна операція переказу коштів гарантує дотримання принципів ACID. За допомогою Alembic успішно застосовано міграцію бази. Integration-тести з використанням in-memory SQLite підтвердили правильність CRUD-операцій та надійність механізму відкату транзакцій.